INTUNE System-of-Systems: Presentation Spoken Script

DSAA 2026 Presentation Script | Speakers: Radhakrishna Bharuka, Abhang Pawar, Nilesh Dwivedi

SLIDE 1: INTRO (TITLE SLIDE)

■ ~1:00 min | ~130 words

"Good morning, everyone. I am Radhakrishna Bharuka, and presenting along with me today are my co-authors Abhang Pawar and Nilesh Dwivedi from IIIT Dharwad. This work was conducted in collaboration with Anshuman Guha from Johns Hopkins University and Dr. Animesh Chaturvedi from IIT Kharagpur.

Today, we are presenting **INTUNE System-of-Systems**, an event-driven architecture for incremental knowledge distillation in compact Large Language Models.

Knowledge distillation transfers capability from large models to small, low-latency students. However, existing industry pipelines treat distillation as a monolithic batch job: you collect a large dataset, wait, and train once. *Data keeps arriving continuously, but our models do not.*

INTUNE introduces an event-native, streaming architecture that continuously trains and evaluates compact LLMs in incremental checkpoints as new data arrives. Together, we will walk you through our problem statement, methodology, and how INTUNE operates end-to-end."

SLIDE 2: PROBLEM STATEMENT & NOVELTY

■ ~1:45 min | ~200 words

"Today, deploying LLMs in production creates a **deployment squeeze**: 100-billion parameter models offer strong reasoning but are too expensive and slow to serve, while 1-billion parameter compact models are fast but weak. Distillation is the bridge.

However, traditional distillation workflows suffer from five critical bottlenecks:
• **Batch job:** Distillation is treated as a one-time process: collect data, wait, and train once. The model remains static while production data shifts.
• **Polling-native orchestration:** Schedulers check for new data on fixed clock intervals. Naive polling pays dead idle time, while asynchronous polling burns compute on speculative prefetching that may never be needed.
• **Teacher choice is a guess:** Teams default to the biggest model available or rely on LLM-as-a-judge, which introduces positional and self-preference bias.
• **Batch is a black box:** You get one score at the end. Intermediate learning dynamics, transient regressions, and early stopping points remain completely invisible.

Our Novelty: INTUNE replaces polling-based batch workflows with an event-native, incremental design principle. We trigger training dynamically via Change Data Capture, select teachers using deterministic metrics, and expose learning trajectories through sequential checkpoints."

SLIDE 3: METHODOLOGY

■ ~2:00 min | ~230 words

"Our methodology addresses each bottleneck across three interconnected tracks:

1. Empirical Teacher Selection: Rather than assuming scale guarantees quality, we benchmarked a domain-aligned 7B teacher (Alpaca-7B) against a 20B generic model (GPT-OSS-20B) across 4,000 samples. Using a deterministic suite of nine non-LLM metrics, Alpaca-7B significantly outperformed OSS-20B in factual grounding and reduced hallucinations by 66% on context tasks ($p < 0.001$). The winning teacher labels the full 20,000-sample dataset.

2. Incremental Distillation: We compare single-pass Batch training on 20,000 samples against Incremental training across four 5,000-sample checkpoints. After each checkpoint, the adapter is evaluated and saved. Both tracks freeze the Gemma-3-1B student and train a LoRA adapter (rank 16, learning rate 2e-4) under the same core settings.

3. Event-Driven Orchestration: PostgreSQL Change Data Capture monitors incoming data. The moment a 5,000-sample partition is ready, an event streams through Apache Kafka to an Apache Spark threshold aggregator, which immediately signals the task dispatcher. *Against naive polling, checkpoint transition latency drops by 97%, from approximately 150 seconds to 6.5 seconds.*"

SLIDE 4: HOW INTUNE WORKS?

■ **~1:45 min | ~200 words**

"This slide illustrates the four operational pillars of how INTUNE functions end-to-end:

1. System-of-Systems Architecture: INTUNE is decoupled into three loosely coupled layers: Teacher Selection, Event-Driven Orchestration, and the Training/Evaluation Loop. Each layer is independently scalable and swappable without rebuilding the overall pipeline.

2. Three Deltas from Traditional MLOps: First, it trains immediately upon data arrival rather than on fixed-interval polling. Second, the student is checkpointed sequentially for incremental distillation. Third, teacher selection is integrated directly into the system using objective metrics.

3. Fault-Tolerant Event Triggering: PostgreSQL CDC detects completed rows, and Kafka streams the triggers. By combining Kafka consumer-group offsets with an idempotent task dispatcher, checkpoints are never lost and never trained twice after a crash—reducing the human role to monitoring only.

4. Dual-Track Execution: The monolithic batch baseline and the 4-checkpoint incremental pipeline execute under the same orchestration layer. After each checkpoint, evaluation metrics are recorded, and the adapter carries forward into the next update."